Lattice: Signed, Replayable Capability Receipts for Model Runs

Lakshman Turlapati
Full Self Browsing
Email: lakshmanturlapati@gmail.com
Code: https://github.com/fullselfbrowsing/Lattice

***Abstract*: Production systems built on large language models increasingly route work across many providers and modalities, yet the run itself remains opaque. A typical stack records, at best, a free text log that no third party can verify and no operator can replay. We present Lattice, a capability first TypeScript runtime in which every model run emits a *capability receipt*: a canonical, signed, and replayable record of what the system was asked to do, how it routed the work, what constraints it enforced, and what it produced. A receipt is canonicalized with the JSON Canonicalization Scheme (RFC 8785), redacted before signing so that no secret ever enters the signed bytes, and signed with Ed25519 inside a Dead Simple Signing Envelope. Routing is a deterministic and inspectable pure function rather than an opaque model call, output invariants carry terminal semantics so a violated run is never silently retried, and a verifier can reconstruct the run offline and confirm that the recomputed output hash matches the signed receipt. We describe the threat model, the design of the receipt and replay pipeline, and an implementation that ships as two published npm packages at version 1.6.0 with modular entrypoints, 1,482 automated tests across 105 files, a seven provider parity contract, and a content addressed command line tool for reproduction and verification. Our evaluation is assurance by construction: the guarantees are expressed as code level invariants and property tests rather than as a latency benchmark. The result is a small runtime that makes auditability and reproducibility a default property of every multimodal model run.**

***Index Terms*: capability receipts, verifiable replay, LLM orchestration, software supply chain, canonical JSON, digital signatures, reproducibility, multimodal AI**

I. Introduction

Software that calls large language models has moved from prototypes into systems that approve refunds, triage insurance claims, route field operations, and process medical and recruiting documents. These systems combine text with images, audio, documents, structured outputs, and tool calls, and they increasingly spread a single task across several model providers for cost, latency, privacy, or capability reasons. As the stakes rise, two questions follow every run. First, can an operator or an auditor trust the record of what happened. Second, can anyone reproduce that run later, offline, without trusting the party that produced the record. Today the honest answer to both questions is usually no.

The reason is structural. The dominant integration libraries are provider wrappers. They normalize the shape of a chat or generation call across vendors and they provide convenience around tools, memory, and agents. They do not produce a tamper evident record of a run, they do not commit to a deterministic and inspectable account of why one model was chosen over another, and they do not let a third party replay the run and check the result against a signed artifact. A free text log is not a proof. It can be edited after the fact, it does not bind the inputs to the outputs, and it carries no signature that a verifier can check with a public key.

This paper argues that the missing primitive is a per run, signed, replayable *capability receipt*, and that a runtime can make such a receipt a default byproduct of normal operation rather than an after the fact instrumentation task. We build this argument concretely through Lattice, a capability first runtime SDK written in TypeScript. A developer describes the job, supplies any mix of artifacts, declares the desired outputs and policy constraints, and optionally attaches a contract. Lattice then routes the work deterministically, packs context, prepares artifacts for the chosen provider, enforces output invariants, and emits a receipt on every outcome, success or failure.

Lattice provides four guarantees that together make a run auditable and reproducible.

1) **Deterministic and inspectable routing.** The router is a pure function of the capability catalog, the request, and the policy. It uses hard filters, a stable score, and a deterministic tie break, and it reports the full set of candidates, the rejected candidates, the fallback chain, and a typed list of reasons when no route is possible. There is no randomness and no model call inside routing.
2) **Terminal output invariants.** Output level tripwires such as "must cite an artifact" or "no personally identifiable information" are evaluated by a pure kernel. A violation produces a terminal error that the fallback chain is forbidden to retry, which prevents a system from quietly trying again until a banned output slips through.
3) **Signed capability receipts.** Every run emits a receipt that is canonicalized with RFC 8785, redacted before signing, and signed with Ed25519 inside a Dead Simple Signing Envelope. The receipt binds the run record to a key through a signature that any holder of the public key can check.
4) **Offline verifiable replay.** A receipt and its content addressed artifacts can be materialized into a replay envelope. Materialization verifies the signature before it loads any artifact, and a property test confirms that reconstructing the run and recomputing the output hash

yields the value committed in the receipt.

We make the following contributions. We define the capability receipt as a signing and replay primitive for multimodal model runs, and we show how a redact then sign ordering and a canonical encoding turn it into a verifiable artifact rather than a log line. We show how a deterministic and inspectable router, terminal output invariants, and a content addressed verification tool compose into a runtime where auditability and reproducibility are default properties. We describe a working implementation that ships publicly with supply chain provenance, modular adoption paths, a seven provider parity contract, and a test suite that encodes the guarantees as invariants. We also show how external executors can mint compatible receipts without adopting Lattice's provider or agent runtime. The remainder of the paper covers background and related work, the threat model and goals, the design, the implementation, the evaluation, a discussion, the limitations, future work, and a conclusion.

II. Background and Related Work

A. Provider wrappers and agent frameworks

A first family of systems normalizes model access. The Vercel AI SDK [1] offers a broad provider agnostic TypeScript toolkit with multimodal generation, tools, and agents. LangChain and LangGraph [2] provide orchestration, sessions, and tracing. LiteLLM [3] is strong routing infrastructure that exposes a common interface across many providers with fallback and gateway features. The OpenAI Agents SDK [4] provides agent loops, sessions, and tool use. The Model Context Protocol [5] standardizes how tools and context attach to a model session. These systems are valuable and widely used. None of them, however, emit a signed and replayable per run receipt. Their tracing is observability for humans, not a cryptographic artifact that a third party can verify against a public key, and their routing is generally either configuration driven or model chosen rather than a deterministic pure function whose full decision is recorded.

B. Software supply chain provenance

A second family secures the provenance of software artifacts. in-toto [6] provides end to end guarantees over a software supply chain by signing the steps that produced an artifact. The Dead Simple Signing Envelope [7] defines a minimal envelope and a pre authentication encoding so that signatures commit unambiguously to a payload and its type. Sigstore [8] makes signing and transparency broadly accessible. SLSA [9] defines levels of supply chain integrity, and The Update Framework [10] addresses survivable key compromise in update systems. Lattice borrows directly from this lineage. It reuses the DSSE envelope and pre authentication encoding, it adopts a key set with explicit key states in the spirit of key rotation and revocation, and it treats the published package itself as a supply chain artifact with provenance attestations. The novelty is to apply this provenance discipline not to a build, but to an individual model run at runtime.

C. Canonicalization and signatures

Signing a structured object requires a deterministic byte encoding, otherwise two semantically equal objects can produce different signatures. The JSON Canonicalization Scheme [11] defines such an encoding for JSON, and the I-JSON message format [12] constrains JSON to an interoperable subset, which matters for how numbers and floats are serialized. Lattice canonicalizes receipts with RFC 8785 and serializes monetary cost as an I-JSON string so that receipts never carry raw floats. Signatures use EdDSA [13], the elliptic curve signature construction that is widely deployed for compact and fast signatures.

D. Transparency and reproducibility in machine learning

A third family documents models and data for accountability. Model Cards [14] and Datasheets for Datasets [15] standardize disclosure about a model or dataset, and reproducibility programs in the research community [16] push toward runs that others can repeat. These efforts describe an artifact or a study. A capability receipt is complementary and finer grained. It describes a single production run and makes that run independently checkable and replayable. Merkle trees [17] provide a standard way to commit to a set of items with a single root, which we identify as the natural mechanism for a future signed lineage root over the artifacts that a run consumed and produced.

E. The gap

Across these families, no system provides a per run, signed, canonical, and offline replayable receipt for multimodal model work, backed by a deterministic and inspectable router and by output invariants with terminal semantics. Lattice occupies that gap.

III. Threat Model and Goals

A. Actors and trust assumptions

We consider four actors. The *issuer* is the Lattice runtime that performs a run and signs a receipt with a private key. The *provider* is an external model service that the runtime calls and that we treat as untrusted and possibly buggy. The *verifier* is any party that later receives a receipt and wishes to check it, possibly without trusting the issuer at the time of verification. The *adversary* may sit between these actors or may hold material that looks legitimate.

B. Adversary capabilities

We assume an adversary who can tamper with a stored receipt, a transcript, or a claimed output after a run completes. We assume an adversary who can attempt to substitute a different model route while presenting a receipt that attests to the intended route. We consider an adversary who has obtained a valid signing key and who then tries to mint a receipt in an older schema in order to bypass integrity fields that newer schemas commit to, a schema downgrade attack. We consider an adversary who replays a valid receipt from a different run, a different relying party, or an earlier time and presents it as

proof of the current run. We also consider a careless issuer whose run inputs contain secrets or personally identifiable information that must never be embedded in a signed and shareable artifact.

C. Security goals

The runtime targets five goals. First, integrity of the run record: a receipt that has been altered must fail verification. Second, non repudiation through signature: a verifier with the public key can confirm which key signed a receipt and whether that key is active, retired, or revoked. Third, redaction before signing: secrets and detected personally identifiable information are removed from the body before the canonical bytes are produced, so they never enter the signature and never travel with the receipt. Fourth, determinism for replay: a verifier can recompute the output hash from the reconstructed run and compare it to the value the receipt commits to. Fifth, downgrade resistance: a receipt in a superseded schema is rejected before any cryptographic work, so possession of a valid key is not sufficient to evade newer schema commitments.

D. Non goals

Several properties are explicitly out of scope. We do not provide confidentiality of provider traffic, which is the responsibility of transport security. We do not reproduce the live behavior of a remote model, since model providers are nondeterministic and outside our trust boundary. Replay reconstructs the recorded run and its output hash, not a fresh call to the provider. A signature alone proves integrity and attribution, not freshness: binding a receipt to a relying party, a nonce, or a validity window requires the optional v1.4 fields and the matching verifier options described under verification below, and a verifier that must accept a receipt only once keeps its own record of accepted content identifiers. Dropping leaf receipts from a receipt tree leaves a smaller tree that still verifies, so tail truncation is detectable only against an out of band count or anchor. We also do not operate a hosted control plane. Lattice is a runtime SDK, and the receipt is a portable artifact that travels with the run rather than a record locked inside a service.

IV. Design

Figure 1 shows the end to end run lifecycle that produces and then consumes a capability receipt. The subsections that follow walk through each stage.

A. The capability first run model

A run is described by a small intent: a `task` string, a set of `artifacts` that may be text, JSON, images, audio, documents, or tool results, a map of declared `outputs` with their schemas, a `policy` that expresses privacy, latency, provider allow and deny lists, and budget, and an optional `contract`. The developer does not select a model. The runtime selects one from a capability catalog and records the decision. This capability first shape is what makes a receipt meaningful, because the receipt can record the declared intent and the chosen route side by side.

B. Deterministic and inspectable routing

Routing is implemented as a pure function, `routeDeterministically`, that takes the capability catalog, the request, and the policy and returns a route decision. It performs no input or output, draws no randomness, and never calls a model. The function first applies hard filters that reject a candidate when the provider is unavailable, when an explicit provider or model override does not match, when a required input or output modality is unsupported, when structured output is required but unsupported, or when the estimated input exceeds the context window. It then applies policy filters that reject on an allow or deny list, on a privacy class, on a no logging or no upload requirement, on a latency class, or on a budget ceiling. Surviving candidates are scored from cost, latency, and context headroom, and ties are broken by a stable order over acceptance status, score, provider name, and model name, with names compared by code unit rather than by locale so the order is the same on every machine. The catalog position is deliberately not part of the score, so the decision does not depend on the order in which providers were registered; a test evaluates every permutation of a near equal cost catalog and requires the same selected route and the same fallback chain. The decision exposes the selected route, all candidates, all rejected candidates, a fallback chain of policy preserving alternatives, and a deduplicated list of typed reasons when no route is possible. The full execution plan is stable JSON with a nine stage pipeline: analysis, transforms, context packing, provider packaging, tool execution, execution, validation, tripwire, and persistence. Because the decision is a pure function and is recorded in full, the receipt can attest to exactly why a model was chosen, and a reviewer can reproduce the decision by re running the function on the same inputs.

C. Output invariants with terminal semantics

Lattice lets a developer attach output invariants through a fluent builder, `inv`, with constructors `mustCite`, `fieldFromTable`, `noPII`, and `matches`, where the last validates against a Standard Schema [18]. These invariants are evaluated by a pure kernel, `evaluateTripwires`, that performs no input or output, reads no clock, and uses no randomness, so the same output and the same invariants always yield the same result. The kernel aborts on the first violation. The default detector set for `noPII` contains four detectors: email, United States social security number, credit card validated by the Luhn checksum, and United States phone number. The kernel scans strings and walks nested objects, arrays, and wildcard paths in a deterministic order, so a value buried inside a structured output is not missed. When a no personally identifiable information invariant fires, the runtime error returned to the caller carries the detector name and the matched substring so the violation can be debugged, and the receipt redaction policy replaces that substring with the

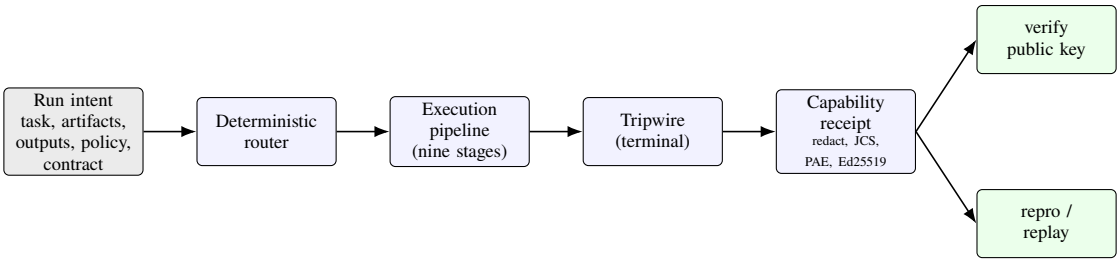


Fig. 1. The Lattice run lifecycle. A run intent is routed by a deterministic and inspectable function, executed through a nine stage pipeline, checked by terminal output invariants, and committed to a signed capability receipt. The receipt is later consumed by offline verification and by content addressed reproduction.

detector name, the match length, and a shape mask before anything is signed; the other three invariant kinds have their observed value reduced to a type label from a closed set in the same step, and every kind's message is rebuilt from the invariant kind and the declared path so that neither model produced object keys nor schema issue text reach the signed bytes. A violation produces a `TripwireViolationError` that carries the flag `terminal: true`. The predicate `isTerminal` returns true for a tripwire violation and for a no contract match, and the fallback chain consults this predicate so that a terminal failure is never retried. This matters for safety: a system that retries until a banned output passes is worse than a system that fails closed, and terminal semantics make failing closed the default.

D. Capability contracts and preflight

A contract expresses requirements that must hold before and around a run: a budget with a maximum cost and, for agent runs, iteration and wall time caps, a set of invariants, a quality floor, a set of required modalities, and a required privacy class. A pure preflight evaluator, `evaluateContractAgainstRoute`, runs inside the router and can reject a route with a typed reason such as a budget exceeded, a missing modality, or a privacy mismatch. A quality floor reason is reserved for the evaluation tool described later. Usage is normalized to prompt tokens, completion tokens, and a cost in United States dollars on every outcome, success or failure, and a null cost is distinct from a zero cost so that an unmeasured price is never confused with a free call.

E. The capability receipt

The receipt is the central artifact. Its construction enforces a strict ordering that is the heart of the security argument. Figure 2 shows the pipeline, and Figure 3 shows the resulting envelope.

First, the runtime assembles the raw receipt body from the run record. Second, it redacts the body. Redaction rewrites the tripwire evidence for every invariant kind, replacing a matched personally identifiable value with a detector name, a length, and a shape mask and reducing other observed values to a type label, and records a manifest of redactions, each with a path and a reason, and a signed redaction policy identifier whose default value is `lattice.default.v1`. Each manifest entry is written at the moment of the corresponding rewrite, so the

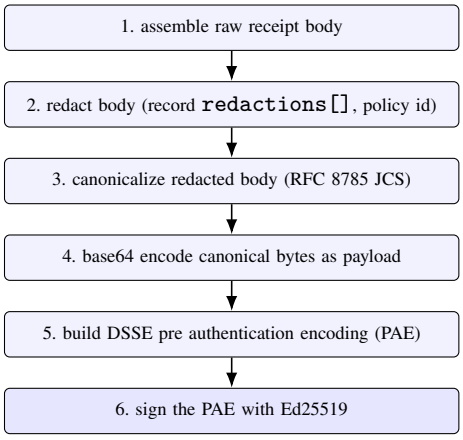


Fig. 2. The receipt construction pipeline. Canonicalization is only ever applied to the output of redaction, so signing unredacted bytes is structurally impossible.

```
{
  "payloadType":
    "application/vnd.lattice.receipt+json",
  "payload": "<base64(JCS(redacted body))>",
  "signatures": [
    { "keyid": "<kid>",
      "sig": "<base64(Ed25519(PAE))>" }
  ]
}
```

Fig. 3. The Dead Simple Signing Envelope produced for a receipt. The pre authentication encoding concatenates the literal `DSSEv1`, the payload type and its length, and the payload and its length, all in a fixed order.

manifest cannot claim a redaction that did not happen. Third, it canonicalizes the redacted body with RFC 8785 [11]. Fourth, it base64 encodes the canonical bytes as the payload. Fifth, it builds the DSSE pre authentication encoding over the payload type and the payload. Sixth, it signs the pre authentication encoding with Ed25519. The ordering is enforced structurally, not by convention: the canonicalization function is only ever called on the output of the redaction function, so a code path that signs unredacted bytes is unrepresentable. This is what we mean by redact then sign as an architectural invariant rather than a guideline.

The signing key is identified by a `kid` and resolved through a key set. A key set maps a `kid` to a key entry whose state is *active*, *retired*, or *revoked*. Cost is serialized as an

I-JSON [12] string or null so that receipts never carry raw floating point numbers. The signature is computed over the pre authentication encoding rather than over the raw JSON, which removes any ambiguity about what the signature commits to.

F. Schema versioning and downgrade defense

Receipts carry an explicit version. Five versions exist: `lattice-receipt/v1`; v1.1, which adds the step marker fields; v1.2, which adds an optional and signed model class field recording the training class of the chosen model; v1.3, which adds the parent receipt content identifier and the artifact lineage root; and v1.4, which adds optional audience, nonce, and expiry fields for freshness binding. New receipts are minted at v1.4 and verifiers accept v1.1 and later. Verification rejects a body whose version is the original v1 or is absent, with the error `schema-version-too-low`, and it does so before any cryptographic work. A verifier that depends on a newer schema's commitments raises the floor with a minimum version option and receives the same error for anything older. This is the downgrade defense from the threat model. An adversary who holds a valid signing key still cannot mint a v1 shaped body to evade the integrity fields that later schemas commit to, because the structural version check fails first.

G. Verification

Verification, `verifyReceipt`, returns a result that is either an ok value with the verified body and the resolved key state, or a failure value with a typed error. There are exactly ten error kinds, listed in Table I. Verification checks the envelope shape, the schema version against the floor, the key existence and state, the re canonicalization match, and the Ed25519 signature, with a defense in depth check that the `kid` inside the body matches the key entry. Only after the signature holds does it apply the optional binding checks: an expected audience, an expected nonce, the receipt's own expiry instant, and a maximum age over the issue time. Running the binding checks last means an unauthenticated envelope cannot be used to probe which audience or nonce a verifier expects. A companion function, `verifyChain`, verifies a set of receipts as a tree linked by parent content identifiers: every link must verify, content identifiers must be unique, all receipts must share one run identifier, exactly one receipt may lack a parent, and every receipt must be reachable from that root.

H. Offline verifiable replay

A receipt becomes useful for reproduction through replay. A replay envelope carries optional receipt and contract fields. The function `materializeReplayEnvelope` reconstructs a replay envelope from a receipt and a set of content addressed artifacts, and it enforces a critical ordering: it verifies the receipt before it loads any artifact. A test confirms that the artifact loader is called zero times when verification fails, which prevents a tampered receipt from causing artifact loads or side effects. Materialization can fail with one of three kinds: `verify-failed`, `artifact-load-failed`, and `envelope-malformed`. A property test asserts that the round trip from `createReceipt` through `materializeReplayEnvelope` through `replayOffline` preserves the output hash, which is the concrete statement of the determinism goal.

TABLE I
THE TEN `VERIFYRECEIPT` ERROR KINDS, IN THE ORDER THEY ARE CHECKED.

Error kind	Fires when
`envelope-malformed`	the envelope cannot be decoded or has no signatures
`version-mismatch`	the body shape or version is unknown
`schema-version-too-low`	the body is v1, has no version, or is below the caller's floor
`key-not-found`	the `kid` is not in the key set
`key-revoked`	the resolved key is in the revoked state
`canonicalization-mismatch`	the re canonicalized body differs from the signed bytes
`signature-invalid`	the Ed25519 check fails or the body `kid` mismatches
`audience-mismatch`	the verifier expects an audience the receipt lacks or differs on
`nonce-mismatch`	the verifier expects a nonce the receipt lacks or differs on
`receipt-expired`	the receipt's expiry instant or the verifier's maximum age has passed

TABLE II
EACH GUARANTEE, ITS MECHANISM, AND ITS ENFORCEMENT.

Guarantee	Mechanism	Enforcement
Deterministic routing	a pure routing function	recorded decision, no I/O or randomness
Terminal invariants	`isTerminal` consulted by fallback	fallback never retries a terminal error
Redact then sign	canonicalize only on redaction output	ordering is structurally unrepresentable otherwise
Verifiable replay	verify before load	loader call count is zero on verify failure; output hash round trip

I. The four guarantees and their enforcement

Table II maps each guarantee to the mechanism that provides it and to the property or test that enforces it. The point of the table is that none of the guarantees rest on a convention or a comment. Each is enforced by structure or checked by a test that fails the build when the property is violated.

V. Implementation

A. Packaging and footprint

Lattice is a TypeScript pnpm workspace that publishes two packages, `@full-self-browsing/lattice` and `@full-self-browsing/lattice-cli`, both at version 1.6.0. The packages are live on the public npm registry. They are published with an OpenID Connect trusted publisher and with supply chain provenance attestations in the spirit of SLSA [9], and a tagged GitHub release accompanies the publish. The runtime targets Node version 24 or newer. The implementation is about 31,100 lines of production TypeScript across 135 non-test source files. The core package has a deliberately small runtime dependency surface of four packages: `canonicalize`

for RFC 8785 encoding, `@standard-schema/spec` for schema contracts, `mime` for artifact typing, and `@noble/ed25519` for the Node 20 audit signing path added in version 1.5.1. The schema library used in tests is a development dependency only, so a consumer of the core package does not inherit it.

Version 1.5.0 adds modular public subpaths for `providers`, `audit`, `context`, `artifacts`, `routing`, `tools`, `storage`, `eval`, `agents`, and `core`. These entrypoints let a downstream runtime adopt only the provider layer, only signed audit artifacts, or only context and artifact preparation. The package still targets Node 24 for the full runtime, while the audit, context, artifact, routing, tools, eval, and core adoption paths are validated for Node 20 style modular consumers.

B. Signing internals

Signing uses the Node 24 WebCrypto interface with the literal algorithm name `Ed25519`. Keys are represented as JSON Web Keys of the octet key pair type and signatures are 64 bytes as specified by EdDSA [13]. To guard against a silent regression in the platform implementation, a development only parity oracle signs the same message with an independent Ed25519 library and asserts a byte for byte match in the test suite. The oracle is never on the production signing path.

C. The seven provider parity contract

Lattice defines seven logical provider adapters: `openai`, `openai-compatible`, `anthropic`, `gemini`, `xai`, `openrouter`, and `lm-studio`. An internal invariant named INV-03 states that every improvement must work equally across all seven universal provider targets. A parity smoke test makes this concrete. As summarized in Table III, the test iterates all seven adapters against provider shaped fake responses and asserts a uniform adapter shape, string raw outputs, the normalized usage shape, an error on a non success response, propagation of an abort signal into the underlying fetch, a raw response pass through, and that the set of adapter identifiers has size seven. The parity contract is what lets a receipt mean the same thing regardless of which provider served the run.

The same provider contract now exposes opt in native provider features for provider only consumers. `ProviderRunRequest.nativeTools`, `nativeToolChoice`, and `nativeStructuredOutput` let an external orchestrator use native tool calling and structured output where adapters support it, without changing the default `ai.run` or agent behavior.

D. The capability registry

Routing consults a capability catalog backed by a registry of 332 model profiles. Of these, 328 are generated from a snapshot of an OpenRouter model feed and 4 are hand maintained static supplements for direct adapters, namely an Anthropic Opus profile, a Gemini Pro profile, an LM Studio local template, and an xAI Grok profile. A weekly continuous integration drift gate guards the generated file so that the catalog does not silently diverge from the upstream feed. At call time the runtime can refine a profile through capability negotiation, fetching a live model listing with a time to live cache and inflight request coalescing, and falling back to the registry on a transient error. Adapter quirk flags capture provider specific behavior so that the runtime can adapt without breaking the parity contract.

TABLE III
WHAT THE INV-03 PARITY SMOKE TEST ASSERTS ACROSS ALL SEVEN PROVIDERS.

Property	Assertion
adapter shape	each adapter exposes the same surface
raw outputs	content is returned as strings keyed by output name
usage shape	normalized prompt and completion tokens and cost
error path	a non success response throws with provider identity
cancellation	the request abort signal reaches the fetch
raw response	the original parsed body is available
identity	the set of adapter ids has size seven

E. External audit and standalone adoption

The receipt layer can also be used without Lattice execution. The `createExternalExecutionAudit` helper accepts externally produced request, response, output, usage, and verdict data, then mints the same signed receipt family, sidecar, replay envelope, and raw request, raw response, input, and output hashes that native Lattice runs produce. Failed verdicts preserve sidecar and receipt evidence but omit replayable outputs, so a failed external execution remains auditable without becoming an offline success fixture.

For runtimes that want preparation without provider execution, `prepareCoreRun` returns artifact references, optional storage references, a context pack, advisory routes, input hashes, warnings, and an execution plan. The tools subpath also exposes MCP resource, prompt, and tool-result artifact helpers, so tool systems can convert their outputs into replayable artifacts without importing the agent runtime.

F. The command line tool

The `lattice` command line tool ships in the second package. It uses a lazily loaded subcommand structure and a consistent exit code convention of 0 for success, 1 for a verification or test failure, and 2 for a load or configuration failure. The `verify` subcommand checks the signature and structure of a receipt and prints the key identifier and contract verdict. The `repro` subcommand verifies a receipt, materializes the replay envelope, replays offline, and diffs the recomputed output hash. The `eval` subcommand walks receipt directories or agent snapshots and gates determinism, cost, and quality regressions relative to a baseline, with a disk backed judge cache. The `receipt diff` subcommand compares signed receipt envelopes and reports structural differences. Together these commands turn a receipt from a passive record into an executable regression and audit gate.

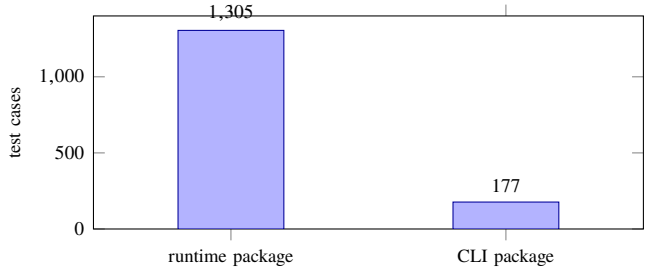


Fig. 4. Automated test cases by package. The suite totals 1,482 cases across 105 files, 1,305 in the runtime package and 177 in the command line package, and it encodes the paper's guarantees as executable obligations.

G. Agents and chained receipts

Lattice exposes a runtime agent entry point, `ai.runAgent`, that drives a uniform tool use loop across all seven providers, and an opt in multi agent crew surface, `defineAgent` and `runAgentCrew`. A crew shares a budget with structural caps on total iterations, per agent iterations, concurrent children, and depth, and it coordinates rate limits through a token bucket with sensible defaults. Crucially, each child receipt chains to a crew root through a parent receipt content identifier, so a multi agent run produces a verifiable tree of receipts rather than a single opaque trace. Each completion receipt commits to that agent's output hash, and `verifyChain`, exposed in the command line tool as `receipt verify-chain`, checks every link, the single root, and the parent linkage of the whole tree. An optional expected count covers the truncation case noted in the non goals, and an optional expected root identifier detects a replaced root. A helper, `evalAgentRun`, gates iterations to goal and cost against a baseline. Three runnable examples exercise these surfaces end to end: a multimodal work inbox that demonstrates a contract, a signed receipt, a tripwire violation, and a no contract match, a single agent loop that emits per iteration checkpoint receipts, and a crew example that demonstrates chained receipts and an evaluation gate.

VI. Evaluation

Our evaluation is assurance by construction. The claims in this paper are properties of the system, and we evaluate them by encoding each as an invariant or a property test that fails the build when the property does not hold, rather than by measuring latency or model output quality, which depend on providers outside our trust boundary. The suite contains 1,482 automated test cases across 105 test files, with 1,305 in the runtime package and 177 in the command line package, and it runs under a standard TypeScript test runner. Figure 4 shows the distribution by package.

We organize the evaluation around the four guarantees.

Determinism of routing and invariants. The router and the tripwire kernel are pure functions. Tests assert that the same inputs always produce the same route decision and the same tripwire result, that every permutation of a near equal cost catalog yields the same selected route and fallback chain, and that the no personally identifiable information walk reports the same violation regardless of key insertion order; the absence of input, output, clock reads, and randomness in these functions is verified by construction and by review. Because the decision is recorded in the plan and the receipt, a reviewer can reproduce a route by re running the function.

Redact then sign. The ordering is structurally enforced because the canonicalization function is only ever called on the output of the redaction function. Tests confirm that the redaction manifest and the signed redaction policy identifier appear in the signed body, that for every invariant kind a sentinel value placed in the evidence does not appear anywhere in the canonical signed bytes, that a no personally identifiable information violation is signed as a detector name, a length, and a shape mask while the runtime error still carries the match, and that redaction is idempotent.

Downgrade resistance and verification. Tests cover all ten verification error kinds, including the audience, nonce, and expiry bindings, the caller raised version floor, and the chain verifier's structural failures. In particular, a body in the original v1 schema, or a body with no version, is rejected with a schema version too low error before any cryptographic work, which demonstrates the downgrade defense. Tests also confirm that a revoked key yields a key revoked error and that a mismatch between the body key identifier and the key entry yields a signature invalid error.

Verifiable replay. A test asserts that the artifact loader is called zero times when receipt verification fails during materialization, which shows that a tampered receipt cannot trigger artifact loads. A property test asserts that the round trip from receipt creation through materialization through offline replay preserves the output hash, which is the determinism goal stated as a checkable property. The `repro` subcommand exercises the same path end to end and returns a distinct exit code for a match and for a drift.

Parity. The INV-03 smoke test exercises all seven providers and asserts the uniform behavior summarized in Table III, including that the set of adapter identifiers has size seven. Parity is what lets a single receipt format and a single verification path apply regardless of which provider served the run.

Taken together, these tests turn the paper's guarantees into executable obligations. A change that weakens a guarantee fails the suite, so the guarantees are maintained across the lifetime of the project rather than asserted once.

VII. Discussion

Signed capability receipts change what is possible after a run. An operator can hand a receipt to an auditor who verifies it with a public key and replays it offline, without access to the original system and without trusting the operator. A dispute about what a system produced becomes a check against a signed artifact rather than an argument about log files. A continuous integration pipeline can use the `eval` subcommand to gate regressions in cost, in determinism, and in quality across a corpus of receipts, which makes a model or prompt change a reviewable event with measurable consequences.

Because the package itself is published with provenance, the chain of trust extends from the source commit through the published tarball to the receipts that the runtime emits.

Three design tensions are worth naming. The first is determinism against model nondeterminism. Models are not deterministic, so Lattice does not promise to reproduce a live call. It promises to reproduce the recorded run and to bind the inputs to the output hash, and it offers a layered determinism check in the evaluation tool for free form text. The second is redaction against reproducibility. A receipt that redacts a secret cannot reproduce a run that depended on that secret value, so redaction is scoped to material that must never be shared, and the redaction manifest makes the omission explicit and signed. The third is modularity against uniformity. Provider only consumers can opt into native tools and structured outputs, while the default runtime and agent paths preserve their existing portable behavior. We accept this explicit split because parity is what makes a receipt portable.

VIII. Limitations

Several limitations follow directly from the design. Replay reconstructs the run record and the output hash, not a fresh model call, so a receipt is evidence about a past run rather than a way to obtain new model behavior. Semantic determinism for free form text relies on a layered judge in the evaluation tool, which is more expensive and less crisp than an exact hash match. The capability registry is a point in time snapshot refreshed by continuous integration, so a brand new model may be absent until the next refresh, although capability negotiation can refine a profile at call time. The full runtime package still targets Node 24, even though several modular subpaths are validated for Node 20 style consumers. When a configured signer fails, the runtime does not hide the gap: by default the result carries a receipt error field and an event, and an operator can instead configure the run to fail. The default personally identifiable information detectors are regular expressions over four United States and email formats documented in the source; names, addresses, and non United States formats are not detected, and the must cite invariant checks that a citation is present, not that it supports the claim. Finally, external audit helpers can sign and replay evidence produced elsewhere, but they cannot prove that the external executor used the claimed provider unless that executor supplies trustworthy raw envelopes.

IX. Future Work

Several extensions build naturally on the receipt primitive. External audit can become stronger when executors attach provider side attestations or gateway signed envelopes. The run event vocabulary can be exported more broadly through OpenTelemetry [19] so receipts and traces share a vocabulary across organizations. Key management can be hardened with key management service backed signers and rotation policies. Realtime audio and video sessions need a distinct session surface and a session summary receipt, since a bidirectional stream differs structurally from a request and response. Finally, the modular subpaths can be validated against more downstream runtimes so adoption does not require replacing an existing orchestration stack.

X. Conclusion

Production systems built on large language models need runs that an auditor can verify and an operator can reproduce. We presented Lattice, a capability first runtime in which every multimodal model run emits a capability receipt that is canonicalized, redacted before signing, and signed inside a Dead Simple Signing Envelope, backed by a deterministic and inspectable router and by output invariants with terminal semantics. A verifier can check a receipt with a public key and replay the run offline, confirming that the recomputed output hash matches the signed value. The implementation ships publicly with supply chain provenance, a seven provider parity contract, a registry of 332 model profiles, modular adoption paths, and 1,482 tests that encode the guarantees as invariants. By making a signed and replayable receipt a default byproduct of every run, Lattice turns auditability and reproducibility from after the fact instrumentation into properties of the runtime itself.

References

[1] Vercel, "AI SDK," Software, online documentation, 2024, https://sdk.vercel.ai.

[2] LangChain Inc., "LangChain and LangGraph," Software, online documentation, 2023, https://www.langchain.com.

[3] BerriAI, "LiteLLM: Call 100+ LLM APIs in the OpenAI format," Software, online documentation, 2023, https://github.com/BerriAI/litellm.

[4] OpenAI, "OpenAI agents SDK," Software, online documentation, 2024, https://github.com/openai/openai-agents-python.

[5] Anthropic, "Model context protocol (MCP)," Online specification, 2024, https://modelcontextprotocol.io.

[6] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium (USENIX Security)*. USENIX Association, 2019.

[7] Secure Systems Lab and the in-toto project, "Dead simple signing envelope (DSSE) specification," Online specification, v1.0, 2021, https://github.com/secure-systems-lab/dsse.

[8] Z. Newman, J. S. Meyers, and S. Torres-Arias, "Sigstore: Software signing for everybody," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 2022.

[9] Open Source Security Foundation (OpenSSF), "Supply-chain levels for software artifacts (SLSA) framework," Online specification, 2023, https://slsa.dev.

[10] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, "Survivable key compromise in software update systems," in *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*. ACM, 2010.

[11] A. Rundgren, B. Jordan, and S. Erdtman, "JSON canonicalization scheme (JCS)," RFC 8785, Internet Engineering Task Force (IETF), 2020, https://www.rfc-editor.org/rfc/rfc8785.

[12] T. Bray, "The I-JSON message format," RFC 7493, Internet Engineering Task Force (IETF), 2015, https://www.rfc-editor.org/rfc/rfc7493.

[13] S. Josefsson and I. Liusvaara, "Edwards-curve digital signature algorithm (EdDSA)," RFC 8032, Internet Engineering Task Force (IETF), 2017, https://www.rfc-editor.org/rfc/rfc8032.

[14] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji, and T. Gebru, "Model cards for model reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAccT)*. ACM, 2019.

[15] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. D. III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021.
[16] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.
[17] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Advances in Cryptology (CRYPTO)*. Springer, 1987.
[18] The Standard Schema project, "Standard schema specification," Online specification, 2024, https://standardschema.dev.
[19] Cloud Native Computing Foundation, "OpenTelemetry specification," Online specification, 2023, https://opentelemetry.io/docs/specs.